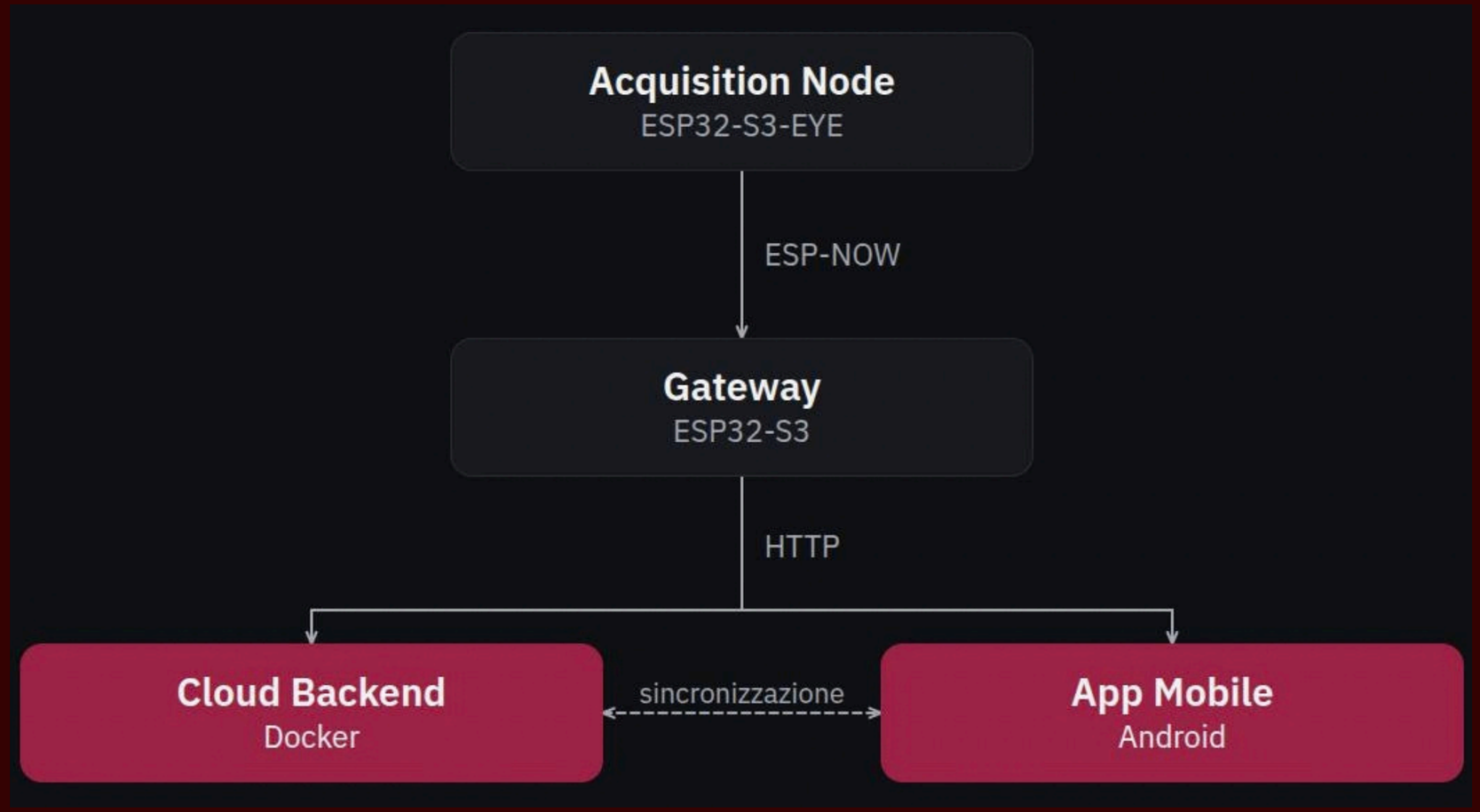
Blood Pattern Analisis

Progetto Cloud and Edge

Emanuele Ugolotti

Progetto a colpo d’occhio



Acquisition Node

Ruolo

Acquisisce un'immagine — o selezionandola da un dataset condiviso, o scattandola davvero con la fotocamera — e la manda al Gateway

Di default, il nodo invia foto in modalità reference, a intervalli regolari.
Il pulsante BOOT aggiunge due azioni, a seconda di come lo premi:
Pressione:
Lunga → Accende / spegne l'invio automatico in reference
Breve → Scatta subito una foto vera

Le due modalità

	Reference	**Captured**
Cosa fa	Sceglie un file da un dataset già prontoSceglie un file da un dataset già pronto	Scatta davvero con la fotocamera
Cosa manda	Solo il **nome del file** (pochi byte)	La foto vera, la comprime in JPEG e la frammenta

Gateway

Come funziona

Riceve tutto quello che arriva dal Node — riferimenti e frammenti di foto — e decide dove inoltrarlo:

1. Controlla se il Cloud è raggiungibile
2. Sì → manda al Cloud
3. No → manda al telefono, come riserva

Se anche il telefono non risponde: i riferimenti vengono messi in coda e riprovati ogni 30 secondi; le foto vere vengono ritentate subito, un paio di volte.

Docker

- **api** (porta 8000) è l'ingresso del Cloud. Riceve le acquisizioni dal Gateway e dal telefono (/cloud/ingest), salva nel database, serve dashboard e risultati (/cloud/results, /cloud/failures, /dashboard) e chiama robustness per classificare.

- **db** (PostgreSQL 16) è l'archivio. Usa l'immagine ufficiale e non c'è codice mio. I dati stanno in un volume nominato (db_data), perciò sopravvivono a docker compose down.

- **robustness** (porta 8001) ospita il modello VGG16 con TensorFlow (3,95 GB) e offre classificazione (/robustness/predict) e attacchi (/robustness/attack). Il modello viene caricato una volta all'avvio, ci mette qualche secondo

- **robustness-simba** (porta host 8002, interna 8001) è l'attacco SimBA,

Perché quattro e non uno

- Un processo per container. Ogni servizio fa una cosa sola e si riavvia da solo.
- Dipendenze molto diverse. L'API è leggera, il modello pesante. Se tutto fosse insieme, ogni modifica all'API ricostruirebbe e riavvierebbe anche il modello.
- Il database è un'immagine ufficiale, non ha senso farla convivere con il nostro codice.
- Scalabilità. Con Swarm potresti dare più repliche all'api e una sola al modello, o spostare robustness su una macchina con GPU.

Swarm

- Stesso sistema, due file: docker-compose.yml per lo sviluppo, docker-stack.yml per Swarm
- Le immagini vanno costruite prima (Swarm non esegue build)
- Swarm non aspetta l'healthcheck: robustness (con compose invece non succede)
- Il riavvio automatico copre questo caso: se un servizio parte troppo presto, riprova

Sviluppi futuri con swarm

- Swarm multi-nodo, con un registry per le immagini
- robustness su una macchina con GPU
- Storage condiviso per foto e dataset

Un nodo solo: ho “dimostrato” il meccanismo, non lo scaling reale. Il percorso verso più macchine è chiaro.

Modelli di DL usati

Perché VGG16, non la pipeline classica

Pipeline classica (RF / SVM)

- Preprocessing complesso: Otsu, filtri morfologici, 25 feature
- Scritta in Python / scikit-image
- Andrebbe riscritta da zero in Java
- Alto rischio di piccole discrepanze che rovinano l'accuratezza

VGG16 (deep learning)

- Conversione diretta in TensorFlow Lite
- Stessa architettura su Cloud e telefono
- Nessuna riscrittura manuale della logica di preprocessing

Stessa Classe predetta, ma confidenza leggermente diversa

Cause Identificate

- Ridimensionamento dell’immagine
- Decodifica JPEG (python e android usano librerie diverse. Queste possono produrre valori dei pixel leggermente diversi già in questa fase)
- Amplificazione attraverso la rete neurale

Perché non è un problema per il funzionamento del sistema

- La classe predetta resta identica tra i due ambienti in tutti i test effettuati
- La confidenza resta comunque nello stesso ordine di grandezza (alta, non ambigua) in entrambi i casi
- Il sistema non basa nessuna decisione architetturale (es. offloading, validazione) sul valore esatto di confidenza, solo sulla classe predetta

Un pulsante e non 4: il conflitto con ADC

Problema: I quattro pulsanti MENU/PLAY/UP/DOWN condividono un unico canale ADC. Crearli — anche con il pacchetto ufficiale — avvia un timer interno che li legge tutti periodicamente, e si va in conflitto con le risorse hardware che la fotocamera usa, causando un crash.

Soluzione: Il pulsante BOOT e' su un pin digitale indipendente, nessun ADC coinvolto. Un solo pulsante fisico: pressione breve = scatta la foto, pressione lunga = attiva/disattiva la modalita' automatica — senza bisogno di riflashare per cambiare comportamento.

Altre decisioni prese

- Le foto restano private, non nella Galleria
- Il dataset e' un volume esterno, non parte dell'immagine
- l'autenticazione è solo sul canale HTTP (gateway↔mobile/cloud), non su ESP-NOW*
- il mdoello e l'attacco ridimensionano l'immagine a 224×224 prima di partire
- l'attacco gira in un Foreground Service, non più dentro il Fragment

* "Non è una differenza di robustezza tecnica tra i due canali — è il modello di minaccia a cambiare: HTTP è esposto a chiunque sulla rete, ESP-NOW è protetto solo dal fatto che estraneo per compromettere la comunicazione deve essere fisicamente presente in nel momento in cui mando la foto."
In altre parole: HTTP è il canale pericoloso perché esposto, ESP-NOW è "sicuro" solo perché nessuno sta davvero provando ad attaccarlo, non perché lo sia davvero sul piano tecnico.

GRAZIE PER L’ATTENZIOEN